PEG SOLITAIRE — SPRINT 4 DOCUMENTATION

---

1. Project Overview

Sprint 4 focused on extending the Peg Solitaire system by adding a **recording and replay mechanism**. The system now allows players to:

- Play manually
- Play automatically
- Randomize the board
- Record gameplay actions
- Save recordings to a text file
- Replay previously recorded games

This sprint emphasized file handling, replay consistency, and preserving game state across sessions.

---

2. Sprint 4 User Stories

ID	User Story
1	Choose board size and type
2	Choose game mode (manual or automated)
3	Start a new game
4	Make manual moves
5	Run automated gameplay
6	Randomize board
7	Detect end of game

8 Record gameplay

9 Replay gameplay

---

3. Acceptance Criteria

User Story 8 — Record Gameplay

AC 8.1 — Record Manual Moves

Given
A user has started a manual game.

When
The user clicks Record and performs valid moves.

Then
All moves are stored in memory.

AC 8.2 — Save Recording

Given
A recording is active.

When
The user stops recording.

Then
The recorded actions are saved into:

game_record.txt

AC 8.3 — Record Randomization

Given
Recording is active.

When
The user clicks Randomize.

Then
The randomize event is recorded.

User Story 9 — Replay Gameplay

AC 9.1 — Replay Recorded Moves

Given
A valid recording file exists.

When
The user clicks Replay.

Then
The game reproduces the recorded moves in sequence.

AC 9.2 — Replay Randomize Events

Given
A recording contains randomize actions.

When
Replay begins.

Then
Randomize events occur in the same sequence.

---

4. Production Code Traceability

User Story ID	Acceptance Criterion	Class Name	Method Name	Status

1	1.1	PegSolitaireGUI	`__init__()`	Complete
2	2.1	PegSolitaireGUI	`__init__()`	Complete
3	3.1	PegSolitaireGUI	`new_game()`	Complete
4	4.1	ManualGame	`attempt_move()`	Complete
5	5.1	AutomatedGame	`make_auto_move()`	Complete
6	6.1	PegSolitaireGUI	`randomize_board()`	Complete
7	7.1	BaseGame	`is_game_over()`	Complete
8	8.1	PegSolitaireGUI	`toggle_record()`	Complete
8	8.2	GameRecorder	`save()`	Complete
9	9.1	PegSolitaireGUI	`replay_game()`	Complete

---

5. Testing Traceability

5.1 Automated / Functional Tests

User Story	AC ID	Test Class	Test Method	Status
4	4.1	Manual Testing	Move validation	Pass
5	5.1	Manual Testing	Autoplay	Pass
6	6.1	Manual Testing	Randomize	Pass
8	8.1	Manual Testing	Record	Pass

9	9.1	Manual Testing	Replay	Pass

5.2 Manual Tests

Test Case	Input	Expected Output	Status
Record Manual Moves	Click Record + Move	Moves stored	Pass
Save Recording	Stop recording	File created	Pass
Replay Moves	Click Replay	Moves replay	Pass
Replay Randomize	Replay recorded randomize	Board randomizes	Pass
Automated Replay	Replay auto game	Moves replay correctly	Pass

---

6. Design Explanation

Sprint 4 introduced a recording and replay architecture.

The project uses object-oriented design:

`PegSolitaire`

Responsible for:

- Board creation
- Move validation
- Move execution
- Peg counting

`BaseGame`

Responsible for:

- Shared gameplay behavior
- Game over detection

`ManualGame`

Responsible for:

- User-driven gameplay

`AutomatedGame`

Responsible for:

- Automatically selecting valid moves

`PegSolitaireGUI`

Responsible for:

- UI rendering
- User interaction
- Button controls
- Replay orchestration

---

7. Sprint 4 Reflection

Sprint 4 introduced persistent gameplay through recording and replay. This required careful synchronization between board state and user actions.

The most challenging part was ensuring replay matched original gameplay exactly, especially when randomization and automated moves were included.

This sprint improved understanding of:

- File I/O
- State persistence
- Event sequencing
- Debugging recorded behavior

---

8. Deliverables

Submitted files:

Production Code

peg_solitaire.py

Recording File

game_record.txt

Sprint Documentation

Sprint4_Report.docx